Entendendo a Utilidade do Root Locus

> Aula de 16/04/2026

Entendendo a Utilidade do Root Locus
- Iniciando atividades e registrando dados
- Iniciando com o Root Locus e suas implicações
  - Planta 1
  - Planta 2
- Trabalho Extra-classe
- Resultados esperados:
- Encerrando seção de trabalho no Matlab
- Material Extra
  - O que é o Root Locus?
  - Resumo sobre Tipos de Respostas Temporais
  - Ganho DC

Iniciando atividades e registrando dados

Antes de inciar com comandos associados com esta aula, vamos estabelecer uma rotina de registro das atividades realizadas no usando o comando `diary` e salvando os dados usando o comando `save` (ao final da seção de trabalho ou final da aula).

A função `diary` indica ao Matlab que seja criado um "dump file", um arquivo texto, contendo a cópia dos comandos usados e respostas obtidas a partir do instante em que foi emitido. Obviamente devemos indicar o nome do arquivo. Então:

```
>> diary aula_16042026.txt
```

> Note que o Matlab cria um arquivo texto, mas eventualmente usando codificação diferente de UTF-8 que é mais atual e normalmente usada com editores texto compatíveis com notação Markdown e criação de páginas HTML. Então, antes de renomear o arquivo .txt criado no Matlab, recomenda-se mudar sua codificação de caracteres para UTF-8 e então renomear o arquivo para .md e editar o mesmo (depois da aula).

Iniciando com o Root Locus e suas implicações

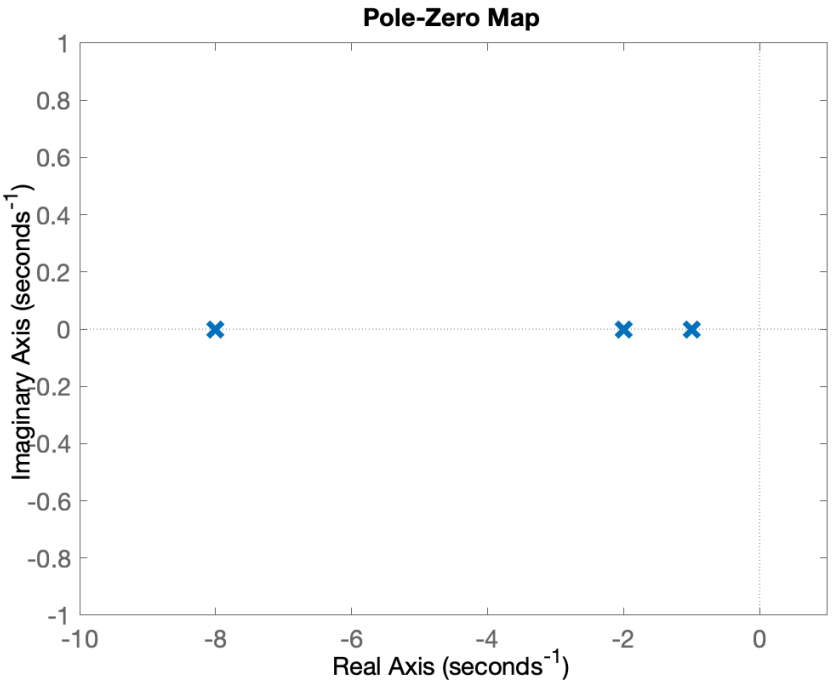
Suponha que queremos analisar alguns sistemas e testar suas respostas para diferentes valores de ganho, especialmente verificar e testar valores de ganho que levem o mesmo para: a) uma resposta rápida mas criticamente amortecido e b) uma resposta rápida mas com algum *overshoot* aceitável (resposta subamortecida).

Planta 1

Seja a planta/sistema:

$$G_1(s) = \frac{1}{(s+1)(s+2)(s+8)}$$

Este é um sistema de 3a-ordem (3 pólos, ou denominador da função transferência com 3 raízes). Ou seja, no plano-s rende algo como:



Ou seja, apenas 3 pólos reais, 2 mais próximos do eixo $j\omega$ e outro mais distante (em $s = -8$, teoricamente menos dominante na resposta).

Ingressando os dados do sistema no Matlab:

```
>> den=poly([- 8 -2 -1]) % criando um polinômio com dados das suas raízes
den =
     1    11    26    16
>> G1=tf(1,poly([-1 -2 -8]))

G1 =

             1
  -------------------------
  s^3 + 11 s^2 + 26 s + 16

Continuous-time transfer function.
```

Podemos comprovar os locais das raízes do denominador fazendo:

```
>> roots(den) % raizes do polinômio
ans =
           -8
           -2
           -1
```

Mas também podemos fazer isto, usando a função `pole()` sobre a função transferência:

```
>> pole(G1)
ans =
            -8
            -2
            -1
```

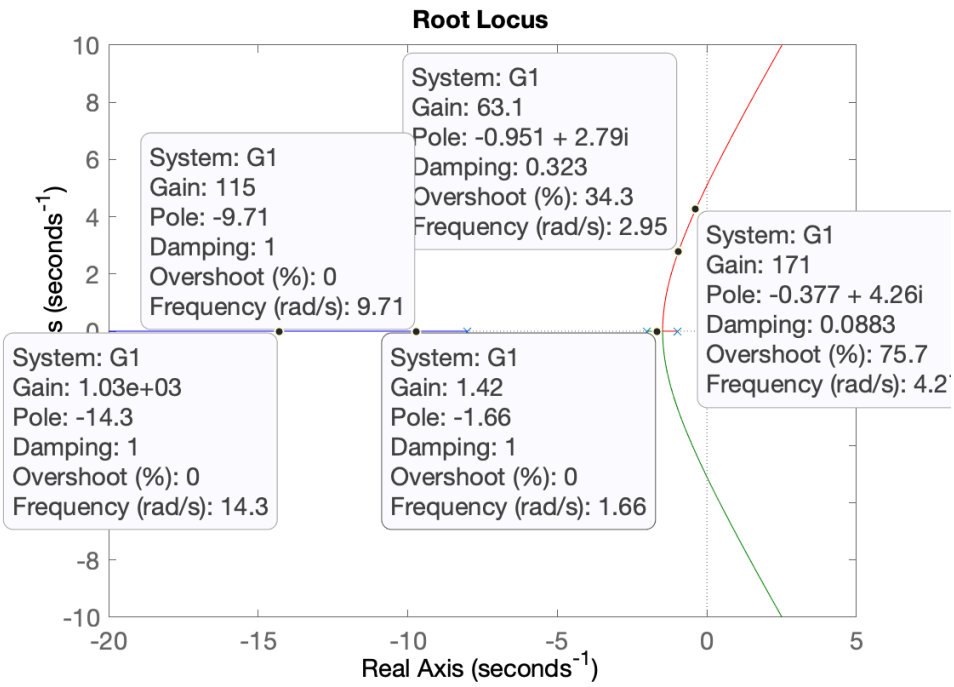
> Por curiosidade o gráfico anterior mostrando a distribuição de pólos e zeros de um sistema pode ser feita no Matlab usando o comando: `pzmap()`. Neste caso, depois de ingressar a *transfer function*, bastaria fazer: `>> pzmap(G1)`.

Por fim, para levantar o gráfico do *Root Locus* para esta planta:

```
>>  rlocus(G1)
```

Para revisar rapidamente teoria sobre Root Locus, ir para [O que é o Root Locus?].

Com o comando `rlocus()` anterior, obtemos os seguintes traçados:



O que o RL nos motra é como evoluem ("caminham") os pólos de malha-fechada a medida que o ganho aumenta.

Testando a resposta do sistema para $K = 1,5$ (a fim de obter uma resposta criticamente amortecida ou super-amortecida):

```
>> K1=1.5;
ftmf1=feedback(K1*G1,1) % fechando a malha

ftmf1 =

             1.5
  --------------------------
  s^3 + 11 s^2 + 26 s + 17.5

Continuous-time transfer function.
```

Para o valor de ganho escolhido, podemos confirmar onde ficaram localizados os pólos de MF:

```
>> pole(ftmf1)
ans =
      -8.0353
      -1.6218
      -1.3429
```

Note que só temos pólos reais, portanto teremos um tipo de resposta do tipo super-amortecida (caso de pólos reais não repetidos) tendendo à uma resposta criticamente amortecida (neste caso, teríamos um par de pólos reais iguais).

Podemos mostrar (sobrepor) no gráfico do RL anterior, os pólos de MF referentes a este valor de ganho, para ajudar a correlacionar a posição dos pólos de MF x resposta temporal obtida:

```
>> hold on % para indicar sobreposição de gráficos
>> plot(real(pole(ftmf1), imag(ftmf1), '+r'))
Undefined function 'imag' for input arguments of type
'tf'.}
```

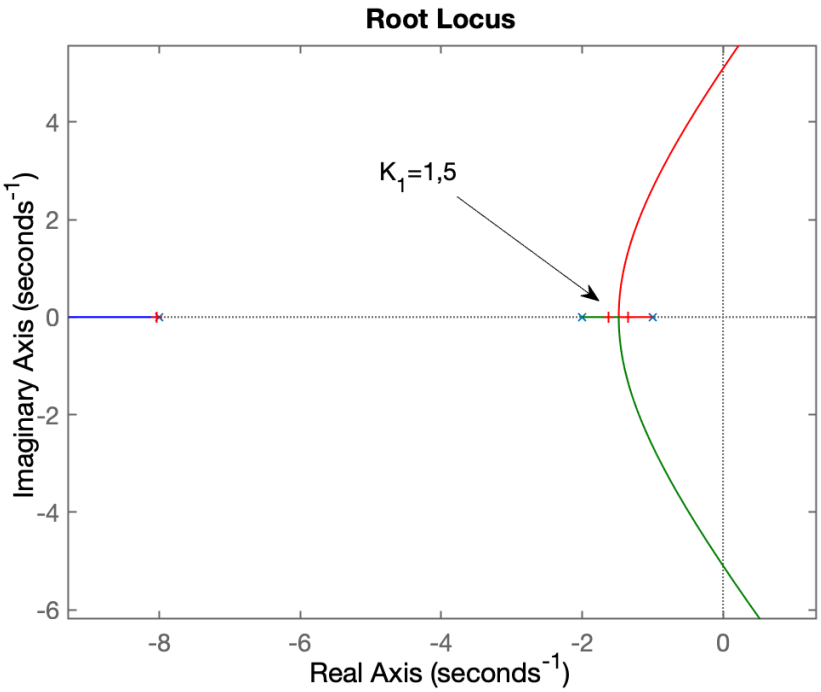
Ops... cometemos algum erro de sintaxe ou de chamada de alguma função. Isto pode ser "consertado" usando comando `help <função>` para tentar descobrir o problema:

```
>> help real
 real   Complex real part.
    real(X) is the real part of X.
    See I or J to enter complex numbers.

    See also help isreal, help imag, help conj, help angle, help abs.

    Other functions named real
```

Hum... observando o comando anterior, notamos que usamos a função `imag` (sintaxe correta) mas sobre uma *transfer funcion* e não sobre uma variável numérica. Este foi o motivo do erro anterior... Ok, consertando o problema:

```
>> plot(real(pole(ftmf1)), imag(pole(ftmf1)), '+r') % plota marcadores "+" vermelhos
(red)
```

Agora sim, podemos observar o RL anterior com as marcas "+" em vermelho ressaltando onde ficam localizados os pólos de MF deste sistema, para o valor de ganho $K = 1,5$:

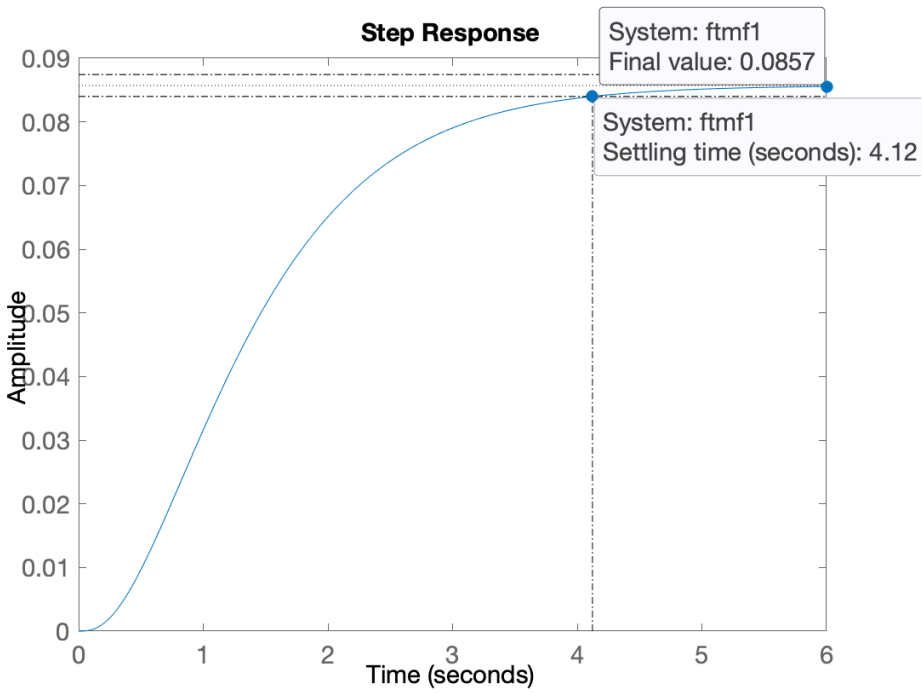


- Mas como seria a **resposta temporal** em malha fechada, deste sistema, com este valor de ganho?

Simples, vamos criar outra janela gráfica, e traçar esta resposta usando a função `step()` do Matlab:

```
>> figure; step(ftmf1)
```

Obtemos então:



Note a resposta quase criticamente amortecida como esperada na teoria vista em [Resposta Temporal de Sistemas Lineares].

Para lembrar de tipos de respostas temporais, ir para: [Resumo sobre Tipos de Respostas Temporais].

Repare também que apesar termos apresentado uma entrada degrau unitário ao sistema de malha-fechada (`step(1*ftmf1)`), o valor final, de regime permanente atingido pelo sistema, foi bem baixo: $y(\infty) = 0,0857$. Isto ocorreu porque a própria função $G_1(s)$ já possui um ganho DC muito baixo, que não aumentou muito mesmo fechando a malha do sistema.

Mais sobre ganho DC ver [Ganho DC].

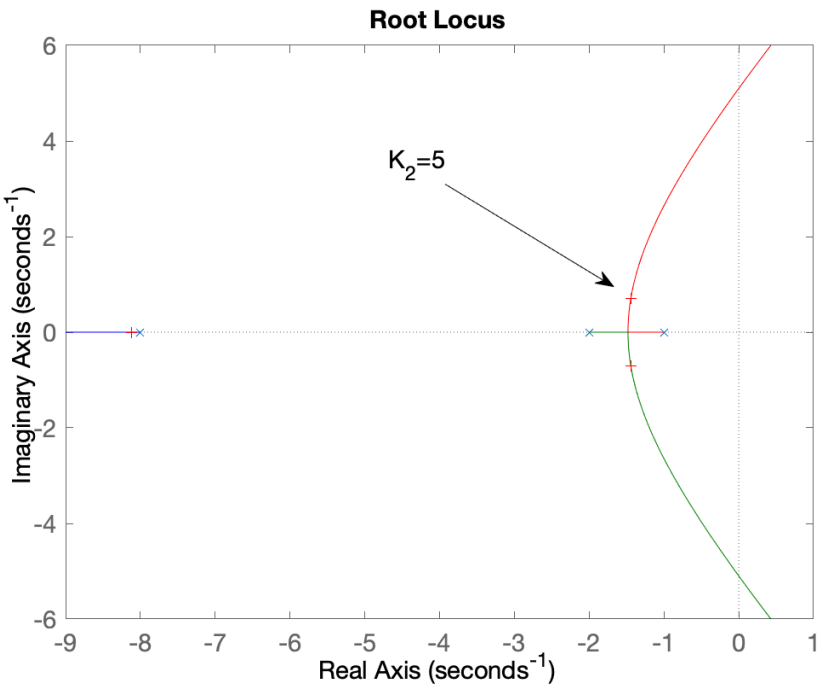
Bom, vamos ver se entendemos o que ocorre quando variamos o ganho no sistema em malha-fechada. Vamos aumentar o valor do ganho e verificar o que ocorre:

```
>> K2=5; % aumentando o ganho para K = 5
>> ftmf2=feedback(K2*G1,1); % calculando novamente a nova MF com este valor de ganho
>> pole(ftmf2) % mostrando onde ficaram localizados estes pólos de MF
ans =
      -8.1149 +          0i
      -1.4425 +    0.71197i
      -1.4425 -    0.71197i
```

Note que agora temos um par de pólos complexos, então uma resposta com *overshoot* (e oscilações) pode ser esperada -- resposta subamortecida.

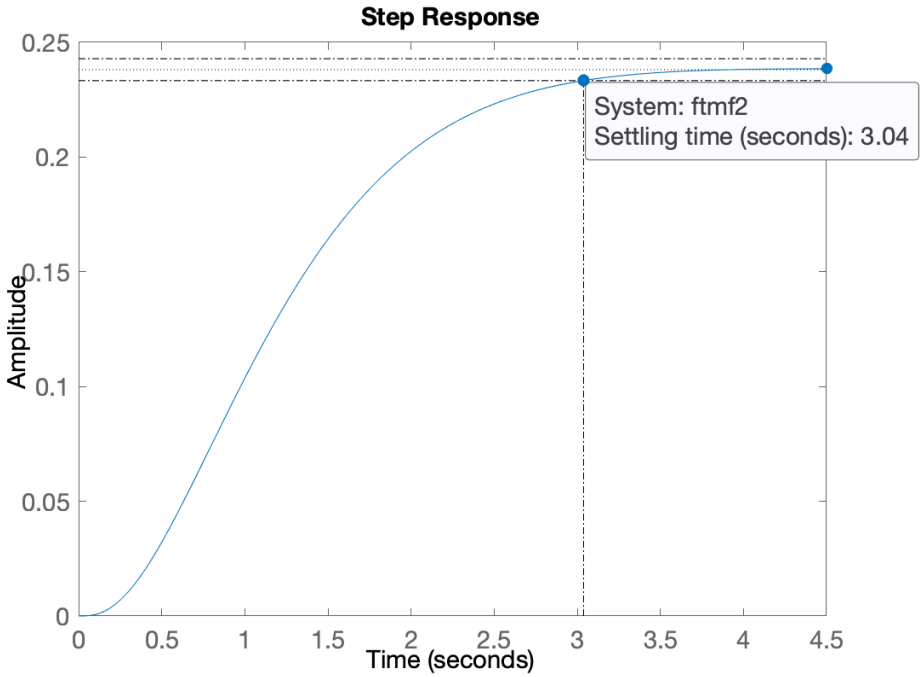
```
>> figure; rlocus(G1)
>> hold on;
>> plot(real(pole(ftmf2)), imag(pole(ftmf2)), '+r')
>> % axis([xmin xman ymin ymax])
>> axis([-9 1 -6 6]) % apenas para restringir a região desejada para o gráfico
```

Obtemos agora este RL:



E observando a resposta temporal que seria obtida então para este valor de ganho:

```
>> figure; step(ftmf2)
```



Apesar dos pólos de MF complexos, a resposta não apresentou um *overshoot* visível.

Mas vamos supor que nosso "cliente", para o qual estamos projetando um sistema de controle automático (em malha fechada) usando apenas controle proporcional, admite um certo valor de *overshoot*. Como podemos descobrir de forma mais rápida o valor de ganho que deveríamos adotar?

Vamos lembrar das equações prevendo parâmetros temporais como $t_s =$ tempo de assentamento, $t_p =$ tempo do instante de pico do *overshoot* e $t_r =$ tempo de subida. Existem outros parâmetros (de sistemas puros de 2a-ordem em MF) como: $\zeta =$ fator amortecimento, $\%OS =$ percentual do *overshoot* e $\omega_n =$ frequência natural da oscilação. Material disponível em: [Resposta Temporal de Sistemas Lineares].

Vamos nos concentrar apenas na equação:

$$\%OS = \exp\left(\frac{-\zeta\pi}{\sqrt{1-\zeta^2}}\right) \times 100\%$$

Esta eq. nos mostra que o *overshoot* depende apenas do parâmetro ζ.

Mas no nosso caso, queremos o inverso, eventualmente nos é dado o valor tolerável máximo para o $\%OS$ e então o que nos falta é saber o valor de ζ. Este valor pode ser encontrado "" invertendo" a eq. anterior e então obtemos:

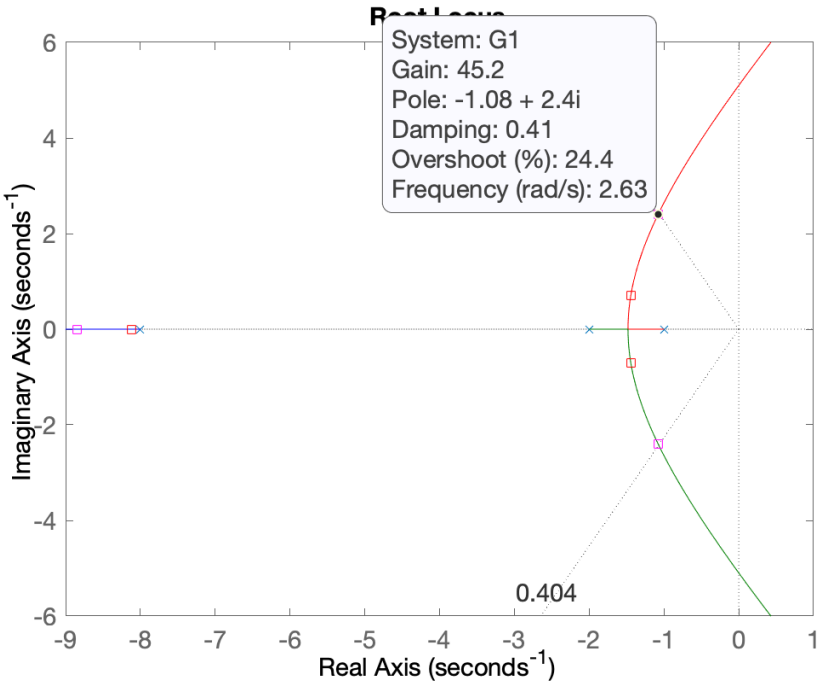
$$\zeta = \frac{-\ln\left(\%OS/100\right)}{\sqrt{\pi^2 + \ln^2(\%OS/100)}}$$

Agora devemos nos lembrar que no plano-s, sistemas com mesmo valor de $\%OS$ ou ζ, devem ter pólos de MF que cruzem com uma linha reta radial no plano-s, ou seja, uma linha reta que parte da origem do plano-s e segue para $-\infty$ e cujo ângulo varia com o valor de ζ.

Ocorre que o Matlab traz uma função que traça (ou sobrepõe) esta linha no plano-s, a função: `sgrid()`. Então podemos traçar um RL e em seguida sobrepor uma linha guia para o ζ ou $\%OS$ desejado, para então descobrir que valor de ganho adotar.

```
>> OS=25; % supondo que seja tolerável um %OS de 25%
>> zeta=(-log(OS/100))/(sqrt(pi^2+(log(OS/100)^2)))
zeta =
      0.40371
>> figure; rlocus(G1)
>> hold on;
>> sgrid(zeta,0) % sobrepondo a linha guia para %OS = 25%
>> axis([-9 1 -6 6]) % restringido a região mostrada do gráfico do RL
```

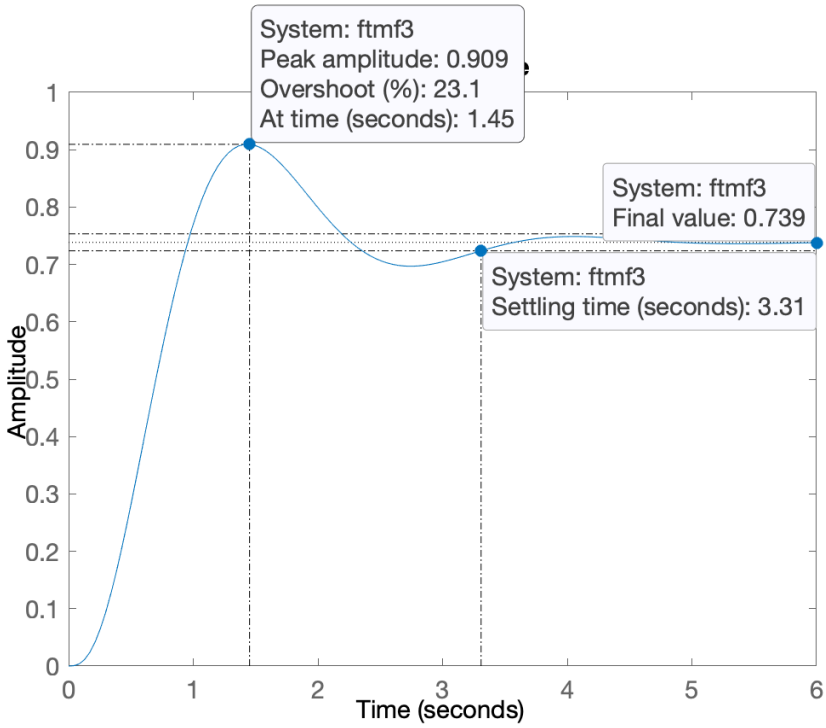
E agora com ajuda do mouse e Data Tips do Matlab, podemos encontrar o correspondente valor aproximado do ganho:



Fechando a malha com este valor, teremos:

```
>> K3=45.2; % fixando o ganho neste valor
>> ftmf3=feedback(K3*G1,1); % fechando a malha
>> pole(ftmf3) % apenas conferindo os pólos de MF
ans =
      -8.8423 +          0i
      -1.0788 +     2.3995i
      -1.0788 -     2.3995i
>> plot(real(pole(ftmf3)), imag(pole(ftmf3)), 'sm') % sobrepondo os mesmos no RL
>> figure; step(ftmf3) % resposta temporal para entrada degrau unitário
```

E então obtemos a seguinte resposta:

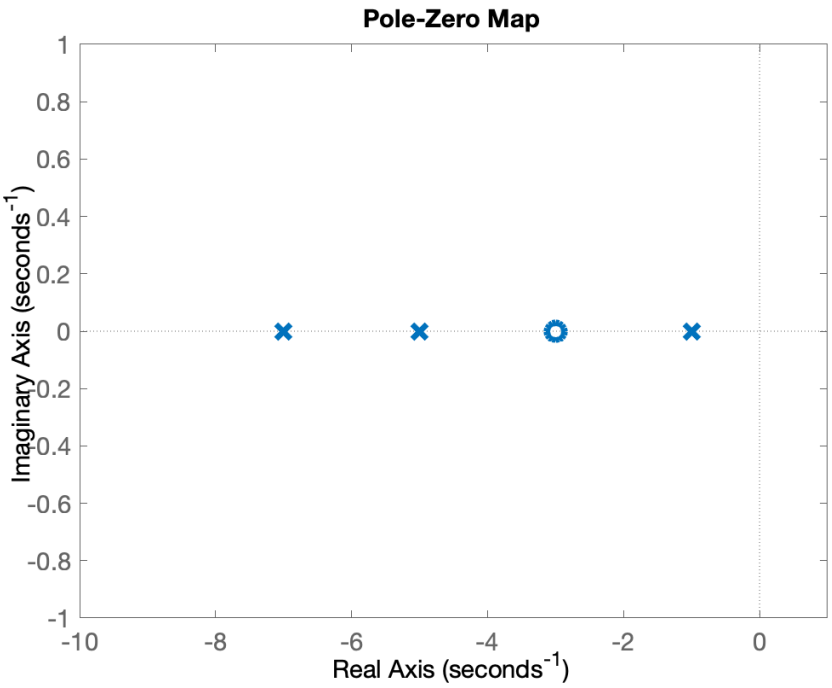


Planta 2

Seja agora outra planta:

$$G_2(s) = \frac{(s+3)}{(s+1)(s+5)(s+7)}$$

Esta planta possui a seguinte distribuição de pólos-zeros no plano-s:



O detalhe é que não estamos controlando o ganho DC da planta, isto é, como no caso de $G_1(s)$, o ganho DC final pode ser muito baixo. A fim de evitar um ganho DC muito baixo, vamos recalcular alguns fatores:

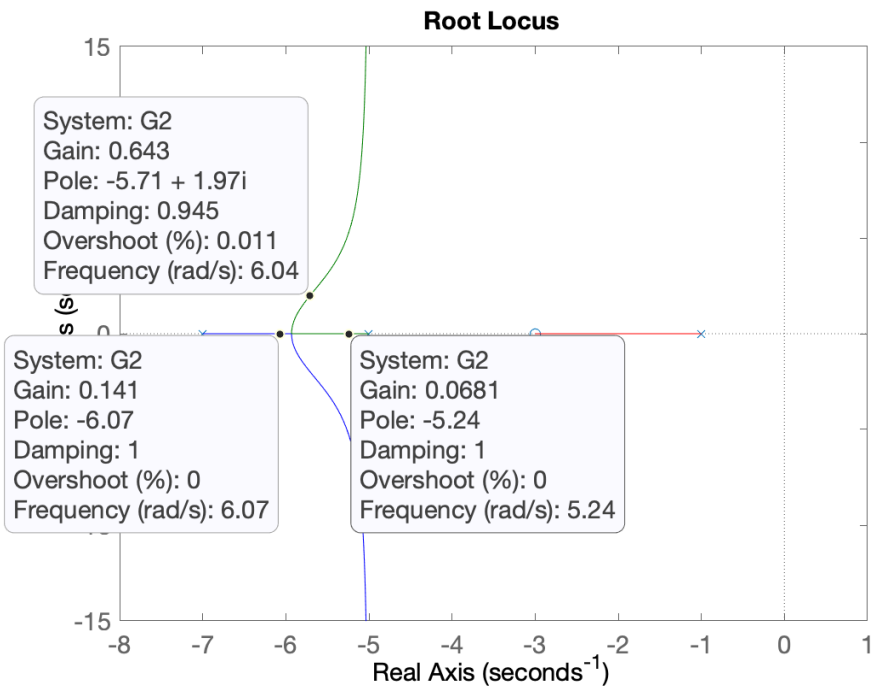
$$G_2(s) = \frac{(1)(5)(7)}{(3)} \cdot \frac{(s+3)}{(s+1)(s+5)(s+7)} = \frac{35}{3} \cdot \frac{(s+3)}{(s+1)(s+5)(s+7)} = \frac{11,6666666667(s+3)}{(s+1)(s+5)(s+7)}$$

Ingressando no Matlab:

```
>> G2=tf(1*5*7*[1 3],3*poly([-1 -5 -7]))

G2 =

           35 s + 105
----------------------------
  3 s^3 + 39 s^2 + 141 s + 105

Continuous-time transfer function.

>> zpk(G2)
ans =

    11.667 (s+3)
-----------------
  (s+7) (s+5) (s+1)

Continuous-time zero/pole/gain model.
```

Verificando como ficaria o RL para este sistema/planta com alguns valores diferentes de ganho:

```
>> close all % limpar janelas gráficas
>> rlocus(G2)
```



Vamos tentar primeiramente uma resposta **criticamente amortecida**, isto é, com pólos reais iguais no plano-s, o que pode ser obtido com $K \cong 0,25$:

```
>> K1b=0.25;
```

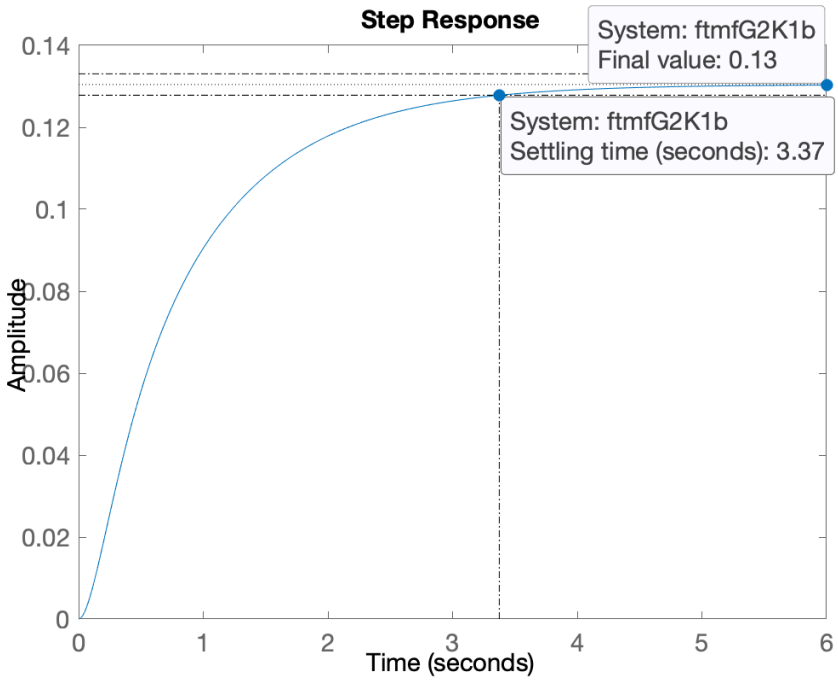
```
>> ftmfG2K1b=feedback(K1b*G2,1) % fechando a malha
ftmfG2K1b =

           8.75 s + 26.25
--------------------------------
  3 s^3 + 39 s^2 + 149.8 s + 131.2

Continuous-time transfer function.

>> pole(ftmfG2K1b) % listando os pólos de MF
ans =
      -5.8814 +    0.87936i
      -5.8814 -    0.87936i
      -1.2371 +          0i
```

Notamos que já obtivemos pólos com raízes complexas. Signica que devemos baixar o ganho:

```
>> K1b=0.2; % testando outro valor de ganho para aproximar + os 2 pólos reais dominantes
>> ftmfG2K1b=feedback(K1b*G2,1); % recalculando a nova FTMF(s)
>> pole(ftmfG2K1b)
ans =
      -5.9046 +    0.63778i
      -5.9046 -    0.63778i
      -1.1908 +          0i
>> K1b=0.16; % ainda pólos complexos, testando outro valor
>> ftmfG2K1b=feedback(K1b*G2,1);
>> pole(ftmfG2K1b)
ans =
      -5.9234 +     0.3434i
      -5.9234 -     0.3434i
      -1.1533 +          0i
>> K1b=0.15;
>> ftmfG2K1b=feedback(K1b*G2,1);
>> pole(ftmfG2K1b)
ans =
      -5.9281 +    0.21519i
      -5.9281 -    0.21519i
      -1.1438 +          0i
```

Por fim, mantido este valor de ganho para evitar perder mais tempo testando outros valores. Verificando a resposta obtida:

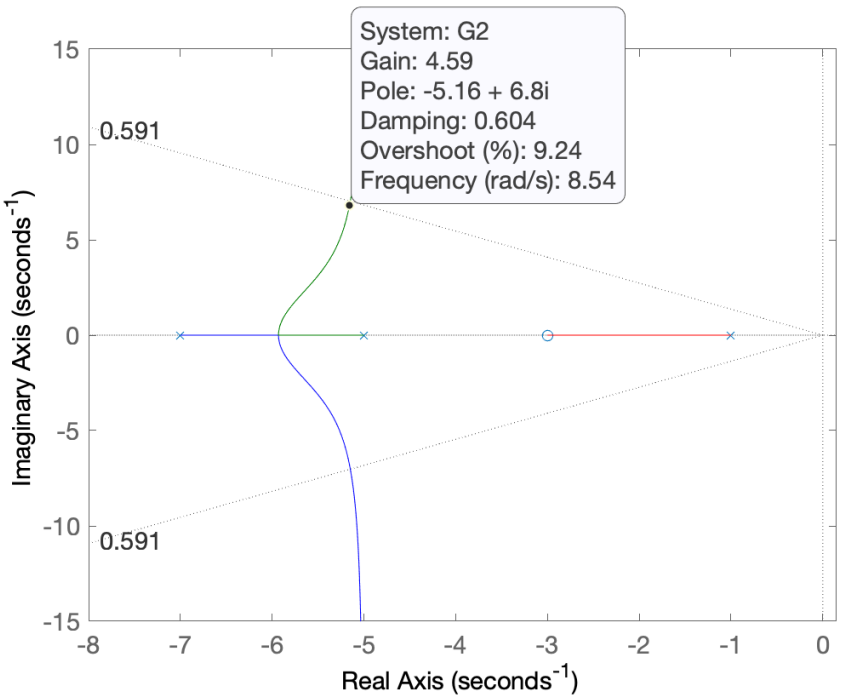
```
>> figure; step(ftmfG2K1b)
```



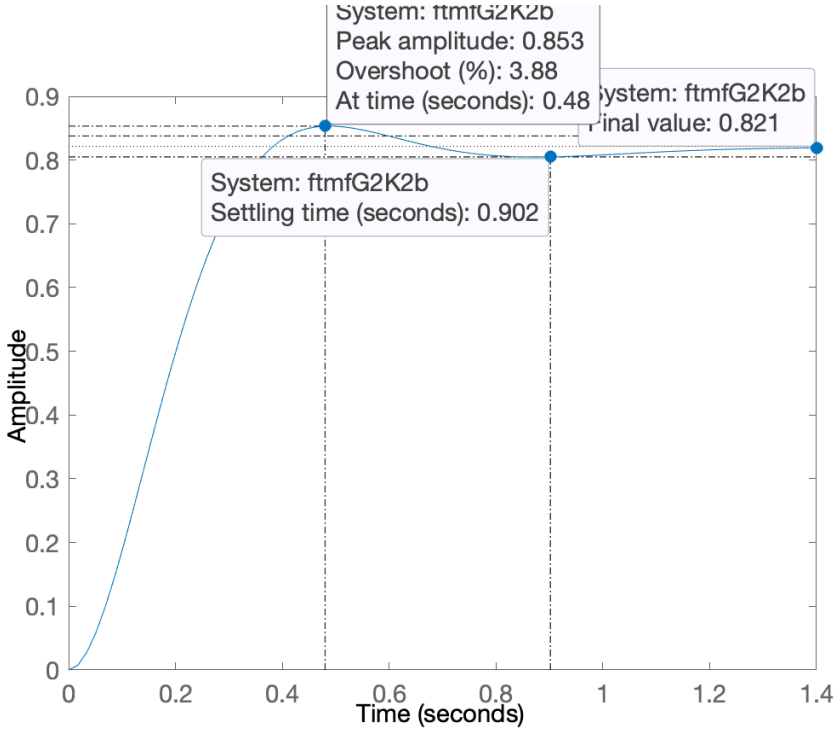
E se objetivo agora é aceitar uma **resposta sub-amortecida**, porém com *overshoot* $\leq$ 10%?

```
>> OS=10; % novo valor para OS
>> zeta=(-log(OS/100))/(sqrt(pi^2+(log(OS/100)^2))) % recalculando fator amortecimento
zeta =
      0.59116
>> figure(1) % fasendo foco na janela da figura contendo ultimo RL
>> hold on
>> sgrid(zeta,0)
```

Buscando no RL o valor do ganho para $\%OS \cong 10\%$:



```
>> K2b=4.6; % ganho para OS aprox 10%
>> ftmfG2K2b=feedback(K2b*G2,1);
>> pole(ftmfG2K2b)
ans =
      -5.1568 +     6.8094i
      -5.1568 -     6.8094i
      -2.6864 +          0i
>> % verificando a resposta obtida:
>> figure; step(ftmfG2K2b)
```



Trabalho Extra-classe

Traçar o RL, descobrir os ganhos e verificar as respostas encontradas, para resposta criticamente amortecida e sub-amortecida tolerando *overshoot* $\leq$ 20\% para os seguintes sistemas:

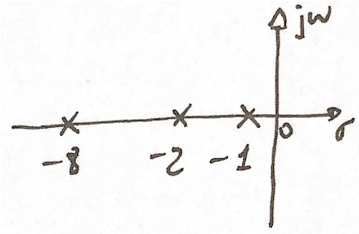
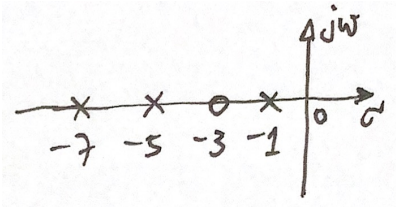
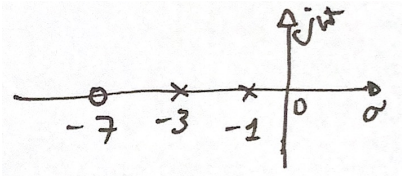
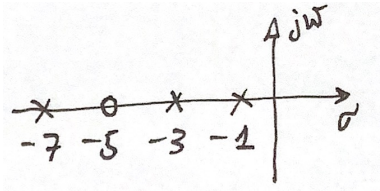
- $G_3(s) = \dfrac{K(s+7)}{(s+1)(s+3)}$
- $G_4(2) = \dfrac{K(s+5)}{(s+1)(s+3)(s+7)}$

Obs.: Determine o valor de K de forma a obter plantas com ganho DC unitário.

Deadline: próxima aula.

Resultados esperados:

O objetivo desta aula era levantar a aprender a usar os **Root Locus** para os seguintes sistemas mostrados abaixo:

Planta	Pólos/zeros no plano-s	Root Locus & Respostas
$G_1(s) = \dfrac{K}{(s+1)(s+2)(s+8)}$		?
$G_2(s) = \dfrac{K\ (s+3)}{(s+1)(s+5)(s+7)}$		?
$G_3(s) = \dfrac{K(s+7)}{(s+1)(s+3)}$		?
$G_4(2) = \dfrac{K(s+5)}{(s+1)(s+3)(s+7)}$		?

Encerrando seção de trabalho no Matlab

Vamos terminar a aula lembrando de fechar o arquivo "*diary*" criado e salvando dados para o caso de querer continuar na próxima aula usando estes dados:

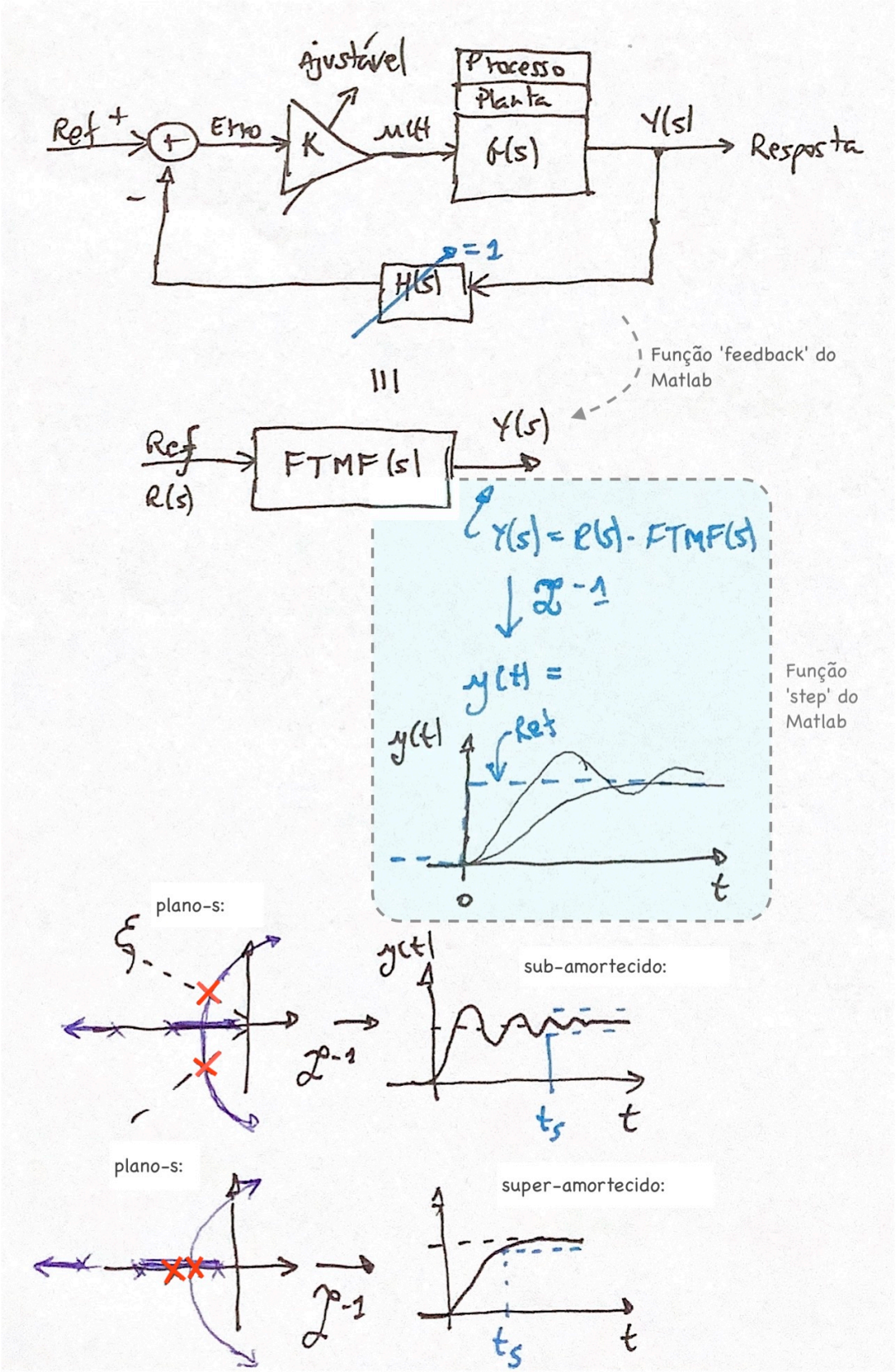
```
>> save aula_16042026 % cria arquivo .mat (binário do Matlab, com dados)
>> diary off % fecha o arquivo diary criado
>> quit % encerra atividades com o Matlab
```

👋

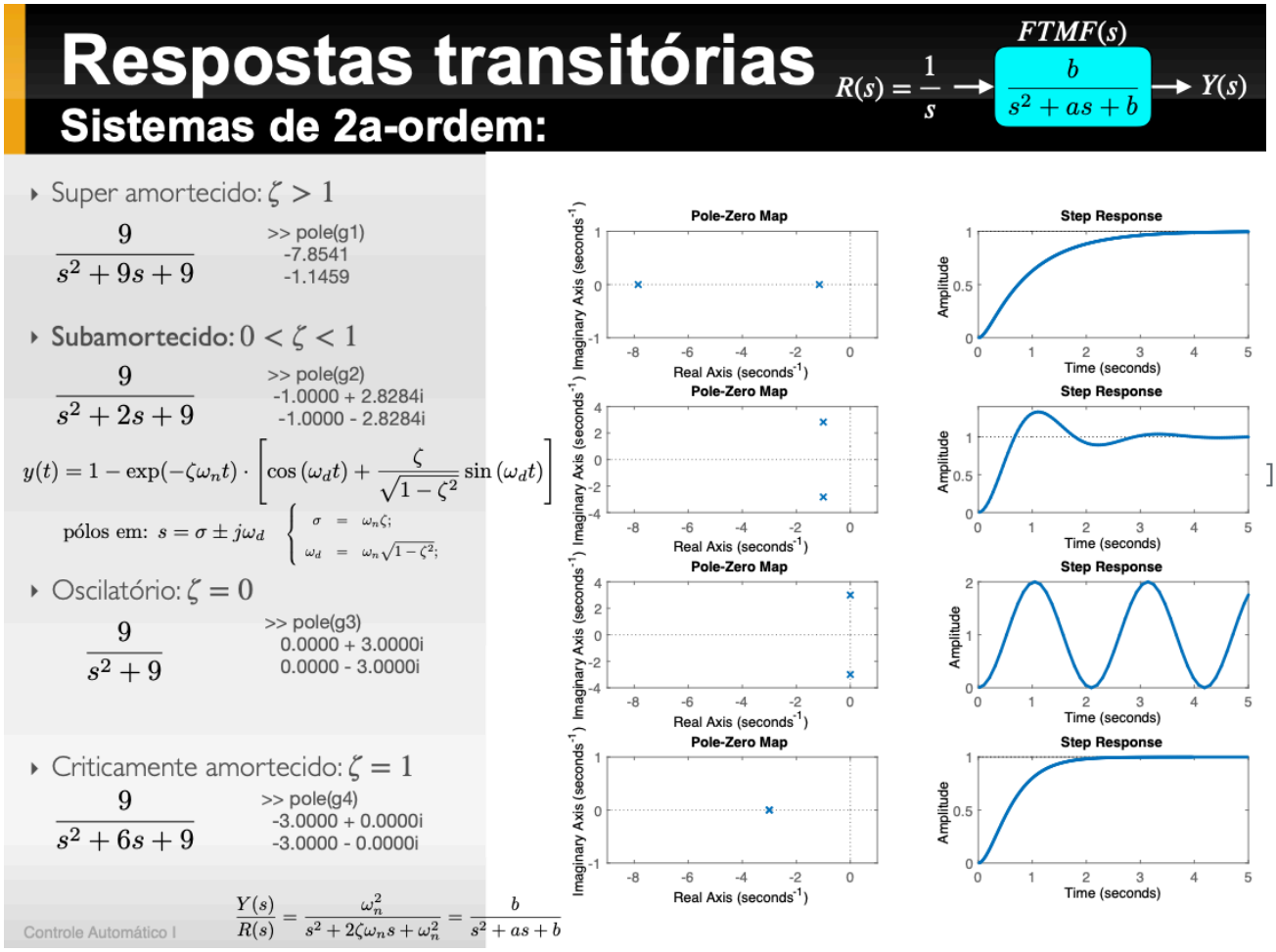
Material Extra

O que é o Root Locus?

Lembre-se: O RL é o resultado do que ocorre quando tomamos a função transferência repassada para o mesmo e o introduzimos dentro de uma malha fechada tradicional, com simples ganho (ou controlador) proporcional K e realimentação negativa. O que é RL faz é justamente variar este valor de ganho e gerar traços no plano-s mostrando como varia a localização dos pólos de MF conforme o valor do ganho K é modificado:



Resumo sobre Tipos de Respostas Temporais



Ganho DC

Como se determina o ganho DC de um sistema?

Resposta: usando o **Teorema do Valor Final** associado com a Transformada de Laplace que estabelece que:

Se $Y(s) = R(s) \cdot FTMF(s)$, então

$y(t) = \mathcal{L}^{-1}\{Y(s)\}$,

Mas podemos determinar $y(t)$ para $t \to \infty$, ou $y(\infty)$, calculando:

$\lim_{t\to\infty} y(t) = \lim_{s\to\infty} s \cdot Y(s)$.

Exemplo$_1$: calculando o ganho DC da planta $G_1(s)$ teremos:

$$G_1(s) = \frac{1}{(s+1)(s+2)(s+8)}$$

Então, quando se aplica uma entrada degrau unitário à este sistema temos:

$Y(s) = R(s) \cdot G_1(s)$

onde $R(s) =$ função transferência da entrada Degrau, que é: $\mathcal{L}\{Degrau\} = 1/s$.

teremos como resposta:

$$Y(s) = \frac{1}{s(s+1)(s+2)(s+8)}$$

Levando em conta o teorema do valor final da Transformada de Laplace:

$$\lim_{t\to\infty} y(t) = \lim_{s\to\infty} s \cdot Y(s) = \lim_{s\to 0} \cancel{s} \cdot \frac{1}{\cancel{s}(s+1)(s+2)(s+8)}$$

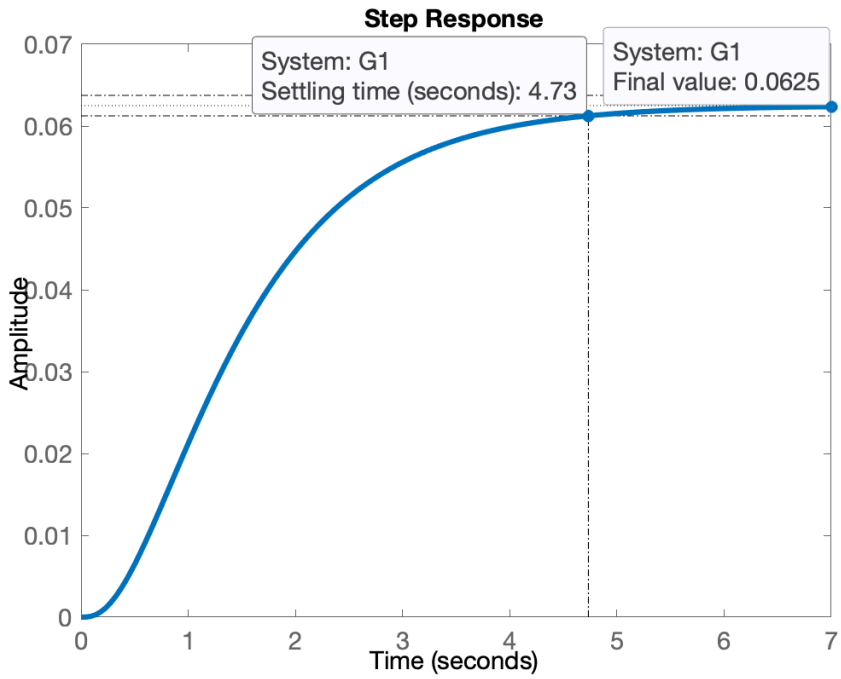
E assim, teremos:

$$y(\infty) = \lim_{s\to 0} \frac{1}{(s+1)(s+2)(s+8)}$$

que resulta:

$$y(\infty) = \frac{1}{(1)(2)(8)} = \frac{1}{16} = 0,0625.$$

Isto significa que o sistema vai tender ao valor de saída 0,0625 quando o mesmo for exposto a uma entrada degrau de valor unitário, ou:



Você pode obter este último gráfico, simplesmente fazendo: `step(G1)`.

Poderíamos ter considerado um ganho DC para a planta $G_1(s)$ fazendo:

$$G_1(s) = \frac{1}{(s+1)(s+2)(s+8)} \cdot \frac{(1)(2)(8)}{(1)} = \frac{16}{(s+1)(s+2)(s+8)}$$

Teste com a função `step()` para comprovar este fato! E depois usando a funçao `dcgain(G1)`.

🎵 Fernando Passold, em 16/04/2026.